MEMORIA - PRÁCTICA 1

Eduardo Guerrero - 816106

~ Inteligencia artificial ~

Introducción

En esta práctica trabajamos con distintos problemas de búsqueda no informada para empezar a familiarizarnos con el lenguaje de programación Python.

Para ello trabajamos en 2 problemas principales:

- Como ampliar algoritmos de búsqueda no informada al 8-puzzle para obtener métricas de rendimiento
- Modelar un problema nuevo como es el de los misioneros y los caníbales.

8-puzzle

> *El 8-puzzle es un juego de mesa para un único jugador que consiste en 8 piezas cuadradas numeradas del 1 al 8 y un espacio vacío en un tablero de 3 x 3. El objetivo del juego es, partiendo de un estado representado por una disposición determinada de las piezas (estado inicial I) realizar movimientos permitidos para alcanzar una configuración deseada (meta O) en el menor número de movimientos posible.*

El objetivo de este ejercicio es para cada algoritmo guardar las métricas que se van usando

Métricas

Para recoger las métricas se ha usado una clase llamada Metrics la cual tiene un atributo por cada métrica que se pide. Todos estos atributos son enteros, inicializados en 0

- Profundidad: Coste de la solución. Se adquiere a través del parámetro depth del nodo solución.
- Expandidos: Nodos que se expanden. se incrementa cada vez que un nodo llama a node.expand(problem).
- Q.Size: tamaño de la frontera en el momento en que se encuentra la solución. Se calcula con la función len
- MaxQS: tamaño máximo que alcanzó la frontera durante toda la ejecución. Se va comprobando cada vez que se modifica la frontera si ha alcanzado el máximo.
  En DLS e IDS no existe una frontera , por lo que se reporta como 0.
- Tiempo: tiempo que tarda el algoritmo. Para medir el tiempo se utiliza time.perf_counter() desde el inicio de la búsqueda hasta el retorno de la función.
- Generados (BFS y UCS): nodos generados, dependiendo del algoritmo se recogen de forma distinta.
  - (BFS y UCS): cada hijo que supera el filtro de no estar explorado ni en la frontera suma uno más.
  - (DLS e IDS): se suma un generado por cada hijo devuelto por expand(), sin filtro de repetidos.

La clase metrics, además, tiene un parámetro llamado error. Este parámetro marca si un algoritmo no es capaz de resolver el problema en un tiempo o memoria razonables.

- error = None → No hay error
- error = 1 → tiempo: se supera un límite de 600 segundos de ejecución.
- error = 2 → memoria: el número de nodos generados supera 2.000.000.
- error = 3 → sin solución: el algoritmo termina sin haber encontrado el estado meta, por ejemplo, cuando DLS devuelve “cutoff”

Estos parámetros se actualizan con una función llamada check_error que comprueba si se han superado los límites.

Resultados

Estos son los resultados que el script muestra al ejecutarse.

Caso	Prof.	Expand	Q.Size	MaxQS	Tiempo (s)	Generados
BFS-G-3	3	4	5	5	0.0001	11
BFS-T-3	3	16	31	32	0.0001	48
DFS-G-3	---	---	---	---	---	(1)
DFS-T-3	---	---	---	---	---	(2)
DLS-3-3	3	4	0	0	0.0001	11
DLS-9-3	9	10	0	0	0.0000	26
IDS-3	3	4	0	0	0.0000	11
UCS-3	3	9	10	11	0.0002	21
BFS-G-9	9	287	198	199	0.0067	487
BFS-T-9	9	16877	29690	29691	0.0599	46568
DFS-G-9	---	---	---	---	---	(1)
DFS-T-9	---	---	---	---	---	(2)
DLS-3-9	---	---	---	---	---	(3)
DLS-9-9	9	5474	0	0	0.0292	15924
IDS-9	9	5474	0	0	0.0225	15924
UCS-9	9	291	201	202	0.0482	493
BFS-G-30	30	181057	365	24048	444.6624	181424
BFS-T-30	---	---	---	---	---	(2)
DFS-G-30	---	---	---	---	---	(1)
DFS-T-30	---	---	---	---	---	(2)
DLS-3-30	---	---	---	---	---	(3)
DLS-9-30	---	---	---	---	---	(3)
IDS-30	---	---	---	---	---	(2)
UCS-30	---	---	---	---	---	(1)

Análisis por algoritmo

Para el análisis he utilizado 3 valores.

- b: factor de ramificación, que para nuestro caso como máximo puede ser 4
- d: profundidad de la solución
- m: la profundidad máxima del árbol de búsqueda, que puede ser infinita si no se controlan los estados repetidos.

Búsqueda en anchura (BFS)

Complejidad teórica: O(b^d) tanto para tiempo como para espacio.
Resulta un algoritmo completo y óptimo cuando el coste por paso es constante.
La diferencia entre grafo y árbol es considerable ya que evitas repeticiones,en BFS-T-3 se generan 48 nodos frente a los 11 de BFS-G-3. Cuando se trata de un problema con solución en 3 pasos no hay problema pero esto se amplifica conforme se aumentan los caminos a solución, en BFS-T-9 ya son 46.568 generados frente a los 487 de BFS-G-9 y en BFS-G-30 se llega al límite de memoria establecido.

Búsqueda en profundidad (DFS)

Complejidad teórica: O(b^d) en tiempo y O(bm) en espacio, ya que solo mantiene en memoria el camino actual y sus hermanos pendientes.
Esto se confirma al ver que DFS-G agota el límite de tiempo (código 1) en los tres casos, no el de memoria. En el caso de DFS-T al no controlar repetidos puede entrar en bucles que hacen que m sea infinito

Búsqueda de profundidad limitada (DLS)

Es equivalente a DFS pero se limita la profundidad a la que se puede llegar.
Complejidad teórica: O(b^límite) en tiempo y O(b·límite) en espacio.
El problema de este algoritmo es que no es completo si el límite es menor que la profundidad real de la solución. Esto ocurre en 3 casos (DLS-3-9, DLS-3-30 y DLS-9-30) donde vemos que el error es (3) ya que el límite impuesto es insuficiente para alcanzar el estado meta. Para el caso de DLS-3-3 y DLS-9-3 vemos que si se llega al estado meta, pero no en la misma profundidad ni con el mismo número de nodos expandidos

Búsqueda iterativa en profundidad (IDS)

Complejidad teórica: O(b^d) en tiempo y O(bd) en espacio.
Coge la completitud y optimalidad de BFS y la combina con el espacio lineal de DFS.
Aquí vemos que los valores de IDS-9 coinciden con los de DLS-9-9 esto ocurre porque IDS va realizando de forma iterativa a DLS, con distintos límites hasta que llega al que le interesa, que justamente es 9.

Búsqueda de coste uniforme(UCS)

Este algoritmo es óptimo cuando los costes entre acciones son distintos, pero en este problema todas las acciones tienen coste 1.
Complejidad teórica: O(b^(1+⌊C*/E⌋)) tanto en tiempo como en espacio, donde C* Es el coste de la solución óptima y E el coste mínimo de cualquier acción. Para nuestro caso como todas las acciones cuestan 1, E = 1 y C* = d. Por lo que la fórmula queda O(b^(1+d))
Espera un comportamiento muy similar a BFS, aunque explorando los nodos en orden de coste acumulado, en lugar de por niveles estrictos

Conclusiones

Como conclusión vemos que las métricas obtenidas reflejan bien la complejidad teórica de cada algoritmo.
BFS y UCS son los únicos algoritmos que garantizan encontrar la solución óptima en las pruebas que se han hecho, pero el tamaño de la frontera y con él el consumo de memoria crecen de manera exponencial con la profundidad de la solución. Por este motivo tanto BFS como UCS acaban fallando en el caso de 30 pasos.

La versión de árbol agrava el problema, ya que no detecta estados repetidos y la ejecución puede quedar atrapada en bucles que nunca terminan.

DLS corrige este problema creando un límite a la profundidad explorada, lo que acota tanto el tiempo como la memoria utilizados, pero introduce un nuevo problema, si el límite elegido es menor que la profundidad real de la solución, el algoritmo termina sin encontrar la solución, aunque exista.

IDS resuelve estos dos problemas, y es el algoritmo que mejor equilibra las dos dimensiones analizadas, alcanza la misma garantía de optimalidad que BFS, pero con un consumo de memoria tan contenido como el de DFS

Misionero y caníbales

El problema de los caníbales y misioneros trata sobre transportar los misioneros y canibales desde un lado del río hasta el otro. Para ello se utiliza un bote el cual puede transportar a 2 personas a la vez. El problema es que en ningún momento puede quedar una zona del rio con mas caníbales que misioneros.

Programación

El problema se ha implementado en P1_canibales.py mediante la clase Canibals, heredada de Problem, siguiendo la representación de estado indicada en el enunciado.

- Estado: tupla (C_izq, M_izq, Bote, C_der, M_der), donde Bote vale 0 si la barca está en la orilla izquierda y 1 si está en la derecha.
- Estado inicial: (3, 3, 0, 0, 0).
- Estado meta: (0, 0, 1, 3, 3).

- Movimientos o acciones: ['1C', '2C', '1C1M', '1M', '2M'], cada una asociada al número de caníbales y misioneros que traslada.

El método actions(state) determina qué acciones son posibles para un estado dado. Primero mira cual es la orilla de origen según el valor del bote y recorre las 5 acciones posibles, descartando aquellas en las que en una de las dos orillas se quedarian mas caníbales que misioneros y que por lo menos un misionero esté en la orilla es decir que si c > m y m > 0 entonces esa acción no es posible para ese estado.

El método result(state, action) calcula el nuevo estado que quedaría al ejecutar la acción a ese estado. En función de donde se encuentre el bote resta los caníbales y misioneras de una orilla y las suma en la otra. Además, cambia el bote de lado modificando su valor.

Los métodos goal_test y el coste por paso se heredan sin modificación de la clase Problem.

Impresión de la solución

La función print_state(state) representa cada orilla escribiendo primero los misioneros “M” y a continuación los caníbales “C” que e encuentre en ella, dependiendo de donde es encuentra el bote dibuja la palabra “BOTE” en un lado u otro de la palabra “--RIO–” .
El recorrido de la solución se obtiene con X.path(), esta función devuelve la lista de los nodos desde el estado inicial hasta la meta, de esta forma, para cada nodo se imprime la acción que lo generó seguida de su estado

```
None |RIBERA-IZQ M M M C C C BOTE --RIO-- RIBERA-DCH
2C |RIBERA-IZQ M M M C --RIO-- BOTE C C RIBERA-DCH
1C |RIBERA-IZQ M M M C C BOTE --RIO-- C RIBERA-DCH
2C |RIBERA-IZQ M M M --RIO-- BOTE C C C RIBERA-DCH
1C |RIBERA-IZQ M M M C BOTE --RIO-- C C RIBERA-DCH
2M |RIBERA-IZQ M C --RIO-- BOTE M M C C RIBERA-DCH
1C1M |RIBERA-IZQ M M C C BOTE --RIO-- M C RIBERA-DCH
2M |RIBERA-IZQ C C --RIO-- BOTE M M M C RIBERA-DCH
1C |RIBERA-IZQ C C C BOTE --RIO-- M M M RIBERA-DCH
2C |RIBERA-IZQ C --RIO-- BOTE M M M C C RIBERA-DCH
1C |RIBERA-IZQ C C BOTE --RIO-- M M M C RIBERA-DCH
2C |RIBERA-IZQ --RIO-- BOTE M M M C C C RIBERA-DCH
```

La solución se encuentra en 11 acciones